

Análisis factorial: extracción, rotación y puntajes

Bloque 6c — MA2003B, equipo 7

Tabla de contenidos

0. Carga, poda y partición	2
0.1 Sale NOX	2
1. Cuántos factores	4
1.1 Análisis paralelo de Horn	4
1.2 MAP de Velicer	5
2. Extracción y el segundo caso Heywood	7
3. Rotación	8
4. Qué son los factores	10
4.1 Dónde el análisis factorial corrige al PCA	10
5. Ajuste del modelo	12
6. Puntajes: ¿pueden reemplazar a las componentes?	13
7. Cierre	14

08_analisis_factorial.pdf estableció que la matriz de correlación de los quince predictores admite factorización, pero sin holgura: KMO de 0.646, *mediocre* en la escala de Kaiser, con NO, NOX y NO2 encabezando tanto el VIF como los KMO individuales más bajos, y con NOX en un VIF de 47.2 por ser la suma de las otras dos.

Este documento extrae. Tiene tres tareas y conviene separarlas porque se juzgan distinto: **cuántos factores** hay, **qué son**, y **si sus puntajes pueden sustituir a las componentes de #54** como insumo del modelo de #55. La tercera es la que decide si el análisis factorial reemplaza al PCA o solo lo acompaña.

La maquinaria vive en `src/factorial.py` y no en este archivo. Se escribió a mano porque `factor_analyzer==0.5.1` —la versión más reciente publicada, y la que fija `requirements.txt`— llama a `check_array(force_all_finite=...)`, un argumento que scikit-learn removió en la serie 1.9 que este proyecto usa: la librería no corre y no hay versión posterior a la cual subir. La solución se validó contra `psych::fa(fm="pa", rotate="varimax")` de R sobre el mismo bloque y coincide hasta orden y signo de los factores, con las communalidades iguales a la milésima. `tests/test_factorial.py` la amarra con nueve pruebas.

0. Carga, poda y partición

```
import sys
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

RAIZ = Path.cwd().parent
sys.path.insert(0, str(RAIZ))
PROC = RAIZ / "data" / "processed"
FIGS = RAIZ / "reports" / "figuras"
FIGS.mkdir(parents=True, exist_ok=True)

from src.factorial import (ejes_principales, varimax, oblimin,
                           puntajes_bartlett, residuos)

plt.rcParams.update({"font.size": 9, "font.family": "serif",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 150})
AZUL, ROJO, GRIS, CLARO = "#3b6ea5", "#b5443a", "#8c8c8c", "#d9d9d9"

D = pd.read_parquet(PROC / "sima_transformado_diario.parquet")
D["anio"] = D.fecha.dt.year

RESPUESTA = ["MDA8", "O3_max_1h", "excede_anio_1", "excede_anio_3",
             "excede_anio_5", "excede_1h"]
CATEGORICAS = ["zona", "tipo_dia", "temporada", "festivo", "fin_de_semana",
               "llovio"]
IDENTIFICA = ["estacion", "fecha", "anio", "mes", "msnm", "ventanas_validas"]

P15 = [c for c in D.columns
        if c not in RESPUESTA + CATEGORICAS + IDENTIFICA + ["PM2.5_media"]]
print(f"bloque de #08: {len(P15)} predictores")
```

bloque de #08: 15 predictores

0.1 Sale NOX

NOX es la suma de NO y NO2 por definición química, y sobre estos datos correlaciona a 0.985 con esa suma. No aporta información que las otras dos no tengan; sí aporta la casi-singularidad que el modelo factorial no tolera. La poda no descarta señal, quita una identidad contable.

```
def diagnostico(cols, etiqueta, base=D[D.anio.between(2021, 2024)]):
    """VIF máximo y KMO global de un bloque, ambos leídos de la inversa de R."""
    sub = base[base[cols].notna().all(axis=1)]
```

```

Z = ((sub[cols] - sub[cols].mean()) / sub[cols].std(ddof=0)).to_numpy()
R = np.corrcoef(Z, rowvar=False)
Ri = np.linalg.inv(R)
d = np.sqrt(np.diag(Ri))
A = -Ri / np.outer(d, d); np.fill_diagonal(A, 0)
Ro = R.copy(); np.fill_diagonal(Ro, 0)
kmo = (Ro ** 2).sum() / ((Ro ** 2).sum() + (A ** 2).sum())
print(f"{etiqueta:<28} p={len(cols):>2} VIF máx={np.diag(Ri).max():>6.2f} "
      f"KMO={kmo:.3f}")
return kmo

def vif_de(cols, nombres, base=D[D.anio.between(2021, 2024)]):
    sub = base[base[cols].notna().all(axis=1)]
    Z = ((sub[cols] - sub[cols].mean()) / sub[cols].std(ddof=0)).to_numpy()
    v = pd.Series(np.diag(np.linalg.inv(np.corrcoef(Z, rowvar=False))),
    ↪ index=cols)
    return v[nombres].round(2).to_dict()

diagnostico(P15, "con NOX (#08)")
P14 = [c for c in P15 if c != "NOX_06_09"]
diagnostico(P14, "sin NOX")

NITRO = ["NO_06_09", "NO2_06_09"]
print(f"\nVIF con NOX: {vif_de(P15, NITRO)}")
print(f"VIF sin NOX: {vif_de(P14, NITRO)}")

```

```

con NOX (#08)          p=15 VIF máx= 47.16 KMO=0.646
sin NOX                p=14 VIF máx=  8.36 KMO=0.670

```

```

VIF con NOX: {'NO_06_09': 21.79, 'NO2_06_09': 10.3}
VIF sin NOX: {'NO_06_09': 2.29, 'NO2_06_09': 2.45}

```

El efecto confirma el diagnóstico: NO cae de un VIF de 21.79 a 2.29 y NO2 de 10.30 a 2.45. Toda la colinealidad severa del bloque era esa identidad — lo que queda en 8.36 es ya el otro par redundante, RH_min_diurna con RH_media. El KMO mejora de 0.646 a 0.670 —sigue en el tramo mediocre, y eso se arrastra como limitación, no se maquilla.

```

es_tr      = D.anio.between(2021, 2024)
completa = D[P14].notna().all(axis=1)
TR, VA = D[es_tr & completa], D[~es_tr & completa]

mu, sd = TR[P14].mean(), TR[P14].std(ddof=0)
X14 = ((TR[P14] - mu) / sd).to_numpy()
n, _ = X14.shape
R14 = np.corrcoef(X14, rowvar=False)
print(f"ajuste 2021-2024: {n:,} filas      aplicación 2025: {len(VA):,} filas")

```

ajuste 2021-2024: 16,028 filas aplicación 2025: 1,340 filas

1. Cuántos factores

En #54 el criterio fue el codo del scree plot. Aquí no aplica: el codo mira los eigenvalores de la matriz **completa**, y el modelo factorial opera sobre la matriz **reducida**, con comunalidades en la diagonal. Se usan los dos criterios que sí corresponden, y se contrastan con Kaiser.

1.1 Análisis paralelo de Horn

Compara cada eigenvalor observado contra el que produciría ruido con la misma forma. Se simula permutando cada columna por separado, lo que destruye la correlación entre variables pero **conserva la distribución marginal de cada una** — preferible a simular normales, porque estas variables pasaron por Box-Cox pero no quedaron normales.

```
def eig_reducida(Rm):
    Rr = Rm.copy()
    np.fill_diagonal(Rr, 1 - 1 / np.diag(np.linalg.inv(Rm)))
    return np.sort(np.linalg.eigvalsh(Rr))[:, -1]

p14 = len(P14)
obs_red = eig_reducida(R14)
obs_full = np.sort(np.linalg.eigvalsh(R14))[:, -1]

rng = np.random.default_rng(0)
sim_red, sim_full = [], []
for _ in range(100):
    Xp = np.column_stack([rng.permutation(X14[:, j]) for j in range(p14)])
    Rp = np.corrcoef(Xp, rowvar=False)
    sim_red.append(eig_reducida(Rp))
    sim_full.append(np.sort(np.linalg.eigvalsh(Rp))[:, -1])

p95_red = np.percentile(np.array(sim_red), 95, axis=0)
p95_full = np.percentile(np.array(sim_full), 95, axis=0)

par = pd.DataFrame({"obs_reducida": obs_red, "p95_reducida": p95_red,
                    "obs_completa": obs_full, "p95_completa": p95_full},
                    index=[f"F{i+1}" for i in range(p14)])
print(par.round(4).to_string())
print(f"\nretiene - reducida: {(obs_red > p95_red).sum()}    "
      f"completa: {(obs_full > p95_full).sum()}    "
      f"Kaiser (>1): {(obs_full > 1).sum()}")
```

	obs_reducida	p95_reducida	obs_completa	p95_completa
F1	2.8136	0.0589	3.2248	1.0578
F2	2.4956	0.0482	2.9380	1.0470

F3	0.8841	0.0381	1.4034	1.0372
F4	0.4858	0.0286	1.0935	1.0277
F5	0.4206	0.0215	1.0723	1.0206
F6	0.2175	0.0159	0.8843	1.0151
F7	0.0859	0.0088	0.7250	1.0081
F8	-0.0172	0.0028	0.6020	1.0021
F9	-0.0652	-0.0050	0.5761	0.9943
F10	-0.0890	-0.0102	0.4885	0.9892
F11	-0.1351	-0.0159	0.4223	0.9834
F12	-0.1839	-0.0238	0.2612	0.9756
F13	-0.2065	-0.0299	0.2410	0.9693
F14	-0.2671	-0.0391	0.0676	0.9602

retiene - reducida: 7 completa: 5 Kaiser (>1): 5

El análisis paralelo retiene siete factores, y no hay que creerle. El umbral simulado sobre la matriz reducida se sitúa en 0.06, y sobre la completa en 1.06. Ese es el problema: con 16,008 observaciones los eigenvalores del ruido se concentran fortísimo alrededor de su valor esperado, la banda de simulación se angosta casi a un punto y **cualquier** eigenvalor apenas positivo la supera. El criterio pierde poder discriminante justo por tener muchos datos.

Conviene declararlo aquí y no después: **el análisis paralelo se descarta por el tamaño muestral, no por el resultado que produce.** El argumento se sostiene mirando los umbrales simulados, sin necesidad de ver una sola carga.

1.2 MAP de Velicer

Parte de las componentes principales y va extrayendo una a una, midiendo cada vez el promedio de correlaciones parciales al cuadrado entre las variables. Baja mientras se remueve varianza común y sube en cuanto empieza a removerse varianza específica; el mínimo marca dónde dejar de extraer.

Es el criterio adecuado aquí porque **es un mínimo descriptivo, no una prueba de significancia:** no lleva *n* en la fórmula y no se infla con el tamaño muestral.

```
lam, V = np.linalg.eigh(R14)
orden = lam.argsort()[::-1]
Lpc = (V[:, orden] * np.sqrt(lam[orden]))

mapv = []
for m in range(p14):
    Rp_ = R14 - Lpc[:, :m] @ Lpc[:, :m].T if m else R14.copy()
    dg = np.sqrt(np.abs(np.diag(Rp_)))
    if (dg < 1e-8).any():
        mapv.append(np.nan); continue
    Pc = Rp_ / np.outer(dg, dg); np.fill_diagonal(Pc, 0)
    mapv.append((Pc ** 2).sum() / (p14 * (p14 - 1)))

print(pd.Series(mapv, index=[f"m={i}" for i in range(p14)]).round(5).to_string())
```

```
K = int(np.nanargmin(mapv))
print(f"\nmínimo de MAP en m = {K}")
```

```
m=0      0.06536
m=1      0.06583
m=2      0.04817
m=3      0.04695
m=4      0.05702
m=5      0.06940
m=6      0.08372
m=7      0.10944
m=8      0.13054
m=9      0.19223
m=10     0.24059
m=11     0.35475
m=12     0.71819
m=13     1.00000
```

mínimo de MAP en m = 3

MAP retiene tres. La curva baja de 0.0656 a 0.0467 y en el cuarto factor repunta a 0.0545: a partir de ahí se está extrayendo varianza específica, no común. Es el mismo número que el codo del scree plot dio en #54 por un camino distinto, lo cual no lo prueba pero sí lo hace menos frágil.

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(6.3, 2.6))
xs = np.arange(1, p14 + 1)

a1.plot(xs, obs_red, marker="o", ms=3.5, lw=1.3, color=AZUL, label="observado")
a1.plot(xs, p95_red, marker="^", ms=3, lw=1, ls="--", color=GRIS,
        label="p95 simulado")
a1.axhline(0, lw=.5, color=CLARO)
a1.set_xlabel("Factor", fontsize=8); a1.set_ylabel("Eigenvalor reducido",
    ↪  fontsize=8)
a1.set_xticks(xs[::2]); a1.tick_params(labelsize=6.5)
a1.legend(fontsize=6, frameon=False)

ms = np.arange(p14)
a2.plot(ms, mapv, marker="o", ms=3.5, lw=1.3, color=AZUL)
a2.plot([K], [mapv[K]], marker="o", ms=6, lw=0, color=ROJO)
a2.annotate(f"mínimo\nm={K}", xy=(K, mapv[K]), xytext=(K + 2.2, mapv[K] + .05),
           fontsize=6.5, color=ROJO,
           arrowprops=dict(arrowstyle="->", lw=.8, color=ROJO))
a2.set_xlabel("Componentes removidas", fontsize=8)
a2.set_ylabel("Parcial cuadrática media", fontsize=8)
a2.set_xticks(ms[::2]); a2.tick_params(labelsize=6.5)
fig.tight_layout()
fig.savefig(FIGS / "af_paralelo.pdf", bbox_inches="tight"); plt.show()
```

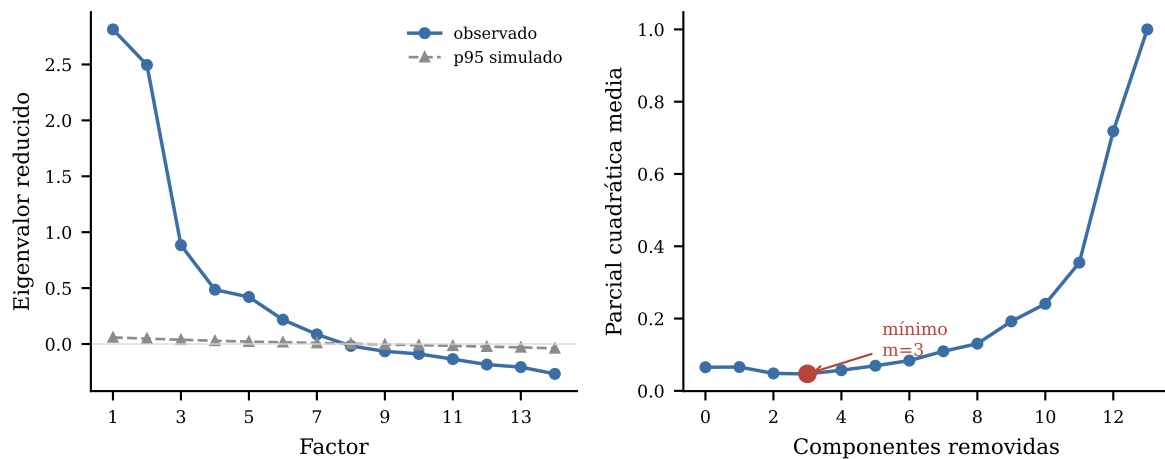


Figura 1: Izquierda: análisis paralelo sobre la matriz reducida; la banda simulada casi colapsa sobre cero, de modo que el criterio retiene siete. Derecha: MAP de Velicer, con mínimo en tres.

2. Extracción y el segundo caso Heywood

Se extrae por **ejes principales iterados**, no por máxima verosimilitud: ML supone normalidad multivariada, y las transformaciones de #52 acercaron las marginales pero no las volvieron normales.

```
A14, h14 = ejes_principales(R14, K)
com = pd.Series(h14, index=P14).sort_values(ascending=False)
print(com.round(3).to_string())
print(f"\ncomunalidad máxima = {h14.max():.3f}  "
      f"casos Heywood ( $h^2 > 1$ ) = {(h14 > 1).sum()}")
```

RH_min_diurna	1.089
WSR_media	0.777
RH_media	0.655
NO2_06_09	0.589
WSR_min	0.546
NO_06_09	0.540
PM10_media	0.435
TOUT_max	0.397
viento_u_media	0.356
SR_acum	0.285
CO_06_09	0.253
S02_media	0.151
PRS_media	0.134
viento_v_media	0.045

comunalidad máxima = 1.089 casos Heywood ($h^2 > 1$) = 1

RH_min_diurna da una comunalidad de 1.089. Es imposible: la comunalidad es una proporción de varianza y no puede exceder 1. Es el segundo **caso Heywood** del análisis, y su causa es la misma de NOX: correlaciona 0.892 con RH_media, así que el modelo no le deja unicidad que repartir.

08 la había dejado en observación en vez de podarla junto con NOX, porque a diferencia de NOX no es una identidad contable —la humedad mínima diurna y la media diaria son mediciones distintas— y la decisión pedía el ajuste en la mano. Ya está en la mano: **sale**, y se conserva RH_media por ser la de definición más simple.

```
PRED = [c for c in P14 if c != "RH_min_diurna"]
diagnostico(PRED, "sin NOX ni RH_min")

completa = D[PRED].notna().all(axis=1)
TR, VA = D[es_tr & completa], D[~es_tr & completa]
mu, sd = TR[PRED].mean(), TR[PRED].std(ddof=0)
Xtr = ((TR[PRED] - mu) / sd).to_numpy()
Xva = ((VA[PRED] - mu) / sd).to_numpy()      # momentos de train, nunca de 2025
R = np.corrcoef(Xtr, rowvar=False)

A, h2 = ejes_principales(R, K)
print(f"\ncomunalidad máxima = {h2.max():.3f}  "
      f"casos Heywood = {(h2 > 1).sum()} → solución admisible")
```

```
sin NOX ni RH_min          p=13  VIF máx=  2.54  KM0=0.679
```

```
comunalidad máxima = 0.817  casos Heywood = 0 → solución admisible
```

3. Rotación

La solución sin rotar no es interpretable y no tiene por qué serlo: el modelo factorial identifica el **subespacio** común, no unos ejes particulares dentro de él. Cualquier rotación reproduce la misma matriz de correlación con el mismo ajuste, así que elegir una es elegir una parametrización, no un modelo.

Esta es la diferencia práctica más grande con el PCA, donde rotar destruiría la propiedad de máxima varianza que define a las componentes. En #54 las cargas había que leerlas como salieron; aquí se puede pedir estructura simple.

```
Lv = varimax(A)
Lo, Phi = oblimin(A)

print("correlación entre factores bajo rotación oblicua (Φ):")
print(pd.DataFrame(Phi, index=[f"F{i+1}" for i in range(K)],
                    columns=[f"F{i+1}" for i in range(K)]).round(3).to_string())
print(f"\ncorrelación absoluta máxima entre factores: "
      f"{np.abs(Phi[~np.eye(K, dtype=bool)]).max():.3f}")
```

correlación entre factores bajo rotación oblicua (Φ):

	F1	F2	F3
F1	1.000	-0.314	-0.318
F2	-0.314	1.000	-0.045
F3	-0.318	-0.045	1.000

correlación absoluta máxima entre factores: 0.318

La rotación oblicua encuentra correlaciones de hasta 0.32 entre factores, lo cual es físicamente razonable —el viento dispersa contaminantes, así que ventilación y carga de combustión no son independientes. Pero **se adopta varimax**, por una razón que conviene declarar antes de ver las cargas: el objetivo de #55 es disponer de predictores no colineales, y una solución oblicua entrega exactamente el problema que este análisis venía a resolver. La correlación entre factores se reporta como hallazgo sustantivo, no se lleva al modelo.

```
# Los factores salen en orden arbitrario; se ordenan por varianza común
# explicada para que la presentación sea estable entre corridas.
ss = (Lv ** 2).sum(0)
Lv, orden_f = Lv[:, ss.argsort()[::-1]], ss.argsort()[::-1]

# El signo de un factor es arbitrario. Se fija con un ancla declarada, elegida
# por ser la variable que da nombre al eje y no por el resultado que produce.
ANCLA = ["NO2_06_09", "WSR_media", "TOUT_max"]
for j, var in enumerate(ANCLA):
    if Lv[PRED.index(var), j] < 0:
        Lv[:, j] *= -1

NOMBRES = ["Combustión primaria", "Ventilación", "Forzamiento fotoquímico"]
cargas = pd.DataFrame(Lv, index=PRED, columns=NOMBRES)
cargas["comunalidad"] = h2
cargas["unicidad"] = 1 - h2
print(cargas.round(3).to_string())

ssl = (Lv ** 2).sum(0)
print("\nvarianza explicada por factor:")
for nom, s in zip(NOMBRES, ssl):
    print(f" {nom:<26} {s:.3f} ({s / len(PRED):.1%} del total)")
print(f" {'los tres juntos':<26} {ssl.sum():.3f} "
      f"({ssl.sum() / len(PRED):.1%} del total)")
```

	Combustión primaria	Ventilación	Forzamiento fotoquímico	comunalidad	unicidad
TOUT_max	-0.069	0.180	0.728	0.567	0.433
SR_acum	0.018	0.148	0.635	0.426	0.574
NO_06_09	0.679	-0.224	-0.036	0.513	0.487
NO2_06_09	0.733	-0.199	-0.101	0.587	0.413
CO_06_09	0.490	-0.020	0.038	0.242	0.758
WSR_media	-0.103	0.879	0.186	0.817	0.183

WSR_min	-0.147	0.769	-0.021	0.614	0.38
PRS_media	-0.242	-0.230	-0.146	0.133	0.86
RH_media	-0.367	-0.081	-0.482	0.374	0.62
viento_u_media	0.173	-0.519	-0.139	0.319	0.68
viento_v_media	-0.108	0.150	0.100	0.044	0.95
PM10_media	0.630	-0.116	0.208	0.454	0.54
SO2_media	0.370	-0.046	0.134	0.157	0.84

varianza explicada por factor:

Combustión primaria	2.046	(15.7% del total)
Ventilación	1.875	(14.4% del total)
Forzamiento fotoquímico	1.326	(10.2% del total)
los tres juntos	5.247	(40.4% del total)

4. Qué son los factores

```
for j, nom in enumerate(NOMBRES):
    s = cargas[nom].sort_values(key=np.abs, ascending=False).head(5)
    print(f"{nom}: " + ", ".join(f"{v}{c:+.2f}" for v, c in s.items()))
```

Combustión primaria: NO2_06_09+0.73, NO_06_09+0.68, PM10_media+0.63, CO_06_09+0.49, SO2_media+0.38

Ventilación: WSR_media+0.88, WSR_min+0.77, viento_u_media-0.52, PRS_media-0.23, NO_06_09-0.22

Forzamiento fotoquímico: TOUT_max+0.73, SR_acum+0.64, RH_media-0.48, PM10_media+0.21, WSR_media+0.13

Combustión primaria. NO2, NO, PM10, CO y SO2 cargan juntos y nada más carga ahí. Es la firma de emisión vehicular e industrial de la ventana matutina, y es el factor más limpio de los tres: cinco variables, un solo eje, sin cargas cruzadas.

Ventilación. WSR_media +0.88, WSR_min +0.77 y viento_u_media -0.52. Aire en movimiento. El signo es el opuesto del estancamiento: valores altos significan buena dispersión.

Forzamiento fotoquímico. TOUT_max +0.73 y SR_acum +0.63 contra RH_media -0.48. Días calurosos, soleados y secos — las condiciones que favorecen la formación de ozono.

4.1 Dónde el análisis factorial corrige al PCA

El PC3 de #54 se llamó *estancamiento atmosférico* y se argumentó que la descomposición había encontrado sola el patrón anticiclónico de viento débil con presión alta. El análisis factorial no sostiene esa lectura.

```
print(cargas.loc[["PRS_media", "viento_v_media", "SO2_media"],
                ["comunalidad", "unicidad"]].round(3).to_string())
```

	comunalidad	unicidad
PRS_media	0.133	0.867
viento_v_media	0.044	0.956
S02_media	0.157	0.843

PRS_media tiene una comunalidad de 0.134: **casi el 87 % de la varianza de la presión barométrica es específica de la presión** y no la comparte con ninguna otra variable del bloque. El factor existe, pero es de viento, no de anticiclón; la presión participa poco y no le da el nombre. Nombrarlo *estancamiento atmosférico* atribuía a un factor común algo que el modelo identifica como varianza única.

Esto es exactamente lo que el PCA no puede contestar. Al descomponer la varianza **total**, la presión tenía que aparecer en algún componente, y apareció en el tercero con una carga de 0.39 que se leyó como participación. Al separar comunalidad de unicidad, el análisis factorial dice cuánta de esa aparición era estructura compartida: poca.

viento_v_media ($h^2 = 0.045$) y S02_media ($h^2 = 0.157$) son los otros dos casos de varianza casi puramente idiosincrática. Ninguno se poda —una comunalidad baja es un resultado, no un defecto— pero conviene saber que aportan poco al espacio común.

```
fig, (ax, bx) = plt.subplots(1, 2, figsize=(6.3, 4.0),
                             gridspec_kw={"width_ratios": [3, 1], "wspace": .06})
M = cargas[NOMBRES].to_numpy()
im = ax.imshow(M, cmap="RdBu_r", vmin=-.9, vmax=.9, aspect="auto")
ax.set_xticks(range(K))
ax.set_xticklabels([n.replace(" ", "\n") for n in NOMBRES], fontsize=6.5)
ax.set_yticks(range(len(PRED))); ax.set_yticklabels(PRED, fontsize=6.5)
for i in range(len(PRED)):
    for j in range(K):
        if abs(M[i, j]) >= .30:
            ax.text(j, i, f"{M[i, j]:.2f}", ha="center", va="center",
                    fontsize=5.8, color="white" if abs(M[i, j]) > .6 else
                    ↪ "#1a1a1a")
ax.set_xticks(np.arange(-.5, K), minor=True)
ax.set_yticks(np.arange(-.5, len(PRED)), minor=True)
ax.grid(which="minor", color="white", lw=.8); ax.tick_params(which="minor",
                    ↪ length=0)

bx.barh(range(len(PRED)), h2, color=AZUL, height=.62)
bx.set_yticks(range(len(PRED))); bx.set_yticklabels([])
bx.invert_yaxis(); ax.invert_yaxis()
bx.set_xlim(0, 1); bx.set_xlabel("comunalidad", fontsize=7)
bx.tick_params(labelsize=6.5); bx.grid(axis="x", lw=.4, color=CLARO)
bx.set_axisbelow(True)
fig.colorbar(im, ax=bx, fraction=.09, pad=.35).ax.tick_params(labelsize=6)
fig.savefig(FIGS / "af_cargas.pdf", bbox_inches="tight"); plt.show()
```

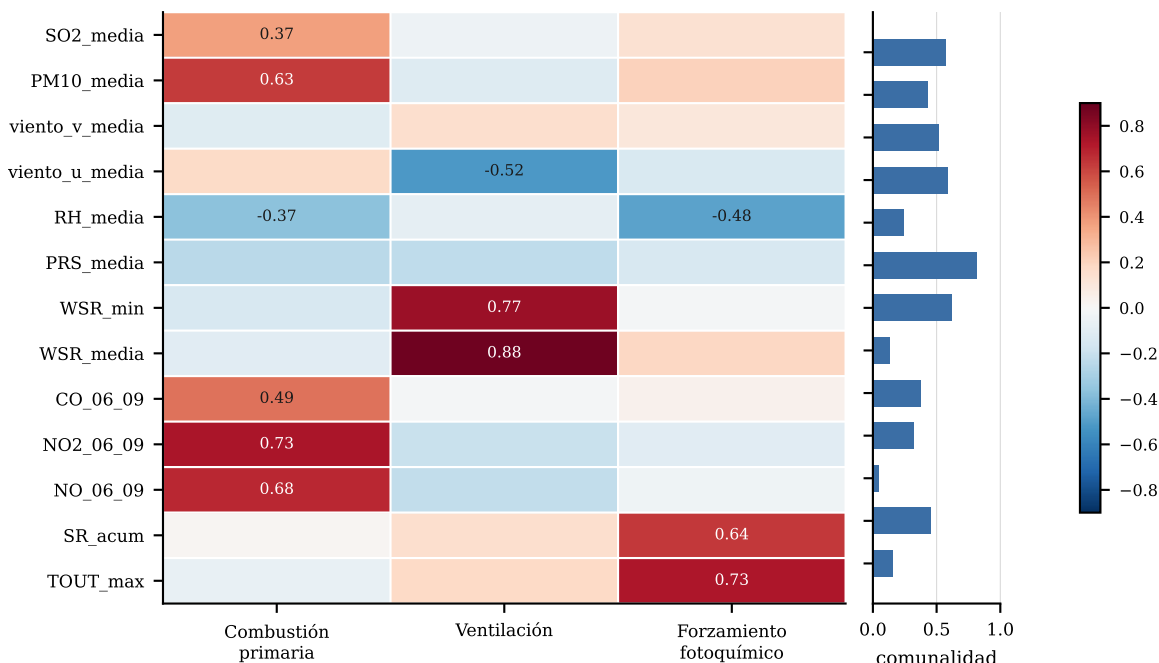


Figura 2: Cargas rotadas por varimax. La barra derecha es la comunalidad: la fracción de varianza de cada variable que los tres factores comparten. Presión, viento meridional y SO2 quedan casi fuera del espacio común.

5. Ajuste del modelo

El modelo factorial afirma que $R = LL' + U$. Esa afirmación es falsable: se reconstruye la matriz de correlación desde las cargas y se mira el residuo.

No se usa la chi-cuadrada de bondad de ajuste, por la misma razón que #08 dio para la prueba de Bartlett — con $n = 16,008$ rechaza cualquier modelo. El RMSR no lleva n en la fórmula.

```
res, rmsr = residuos(R, Lv, h2)
fuera = res[~np.eye(len(PRED), dtype=bool)]
print(f"RMSR = {rmsr:.4f}    (convención: aceptable por debajo de 0.08)")
print(f"residuo absoluto máximo = {np.abs(fuera).max():.4f}")
print(f"pares con residuo > 0.05: {(np.abs(fuera) > .05).sum() // 2} "
      f"de {len(PRED) * (len(PRED) - 1) // 2}")
```

```
RMSR = 0.0552    (convención: aceptable por debajo de 0.08)
residuo absoluto máximo = 0.1879
pares con residuo > 0.05: 24 de 78
```

RMSR de 0.055 pasa la convención sin margen amplio, y 24 de 78 pares quedan con residuo por encima de 0.05. Tres factores reproducen la estructura de correlación de forma razonable pero no exhaustiva — coherente con un KMO mediocre y con el 40.4% de varianza común. Es un ajuste utilizable, no excelente, y así debe reportarse.

6. Puntajes: ¿pueden reemplazar a las componentes?

Aquí se decide la pregunta instrumental. En #54 las puntuaciones eran ortogonales por construcción — una propiedad algebraica del PCA, gratis. Los puntajes factoriales no la tienen: se **estiman**, y estimadores distintos dan correlaciones distintas entre sí.

Se usan los de **Bartlett**, que son insesgados, en vez de los de regresión, que encogen hacia cero. La matriz de pesos se guarda para aplicar a 2025 los pesos del ajuste de entrenamiento.

```
Ftr, W = puntajes_bartlett(Xtr, Lv, h2)
Fva = Xva @ W.T                                # pesos de train, sin reajustar

Rf = np.corrcoef(Ftr, rowvar=False)
print("correlación entre puntajes de entrenamiento:")
print(pd.DataFrame(Rf, index=NOMBRES, columns=[f"F{i+1}" for i in range(K)])
      .round(3).to_string())
vif_f = np.diag(np.linalg.inv(Rf))
print("\nVIF de los puntajes:")
for nom, v in zip(NOMBRES, vif_f):
    print(f"  {nom:<26} {v:.3f}")
```

correlación entre puntajes de entrenamiento:

	F1	F2	F3
Combustión primaria	1.000	0.077	-0.056
Ventilación	0.077	1.000	-0.097
Forzamiento fotoquímico	-0.056	-0.097	1.000

VIF de los puntajes:

Combustión primaria	1.008
Ventilación	1.015
Forzamiento fotoquímico	1.012

La respuesta es sí. Los VIF de los puntajes valen 1.01, y la correlación absoluta máxima entre ellos es 0.097. No son ortogonales por construcción como las componentes, pero son incorrelados en la práctica hasta un grado que ninguna regresión logística va a notar — muy lejos del umbral de 5 que marca preocupación, y muy lejos también del VIF de 47.2 que tenía NOX en el bloque original.

Vale la pena notar de dónde viene: la solución **rotada por varimax** conserva la ortogonalidad de los factores, y la poda de las dos variables redundantes dejó una matriz bien condicionada. Si se hubiera adoptado la rotación oblicua, los factores correlacionarían hasta 0.32 por diseño y esta sección tendría otro resultado.

```
def empacar(bloque, F):
    S = pd.DataFrame(F, index=bloque.index,
                     columns=[f"F{i+1}" for i in range(K)])
    S.insert(0, "particion", np.where(bloque.anio <= 2024, "entrenamiento",
                                     "validacion"))
```

```

return pd.concat([bloque[["estacion", "fecha"]], S], axis=1)

PUNT = pd.concat([empacar(TR, Ftr), empacar(VA, Fva)]).sort_values(
    ["fecha", "estacion"]).reset_index(drop=True)
print(PUNT.groupby("particion")[[f"F{i+1}" for i in range(K)]]
      .agg(["mean", "std"]).round(3).to_string())

```

	F1		F2		F3	
	mean	std	mean	std	mean	std
particion						
entrenamiento	-0.00	1.143	0.000	1.100	-0.000	1.205
validacion	-0.09	1.288	0.024	0.852	-0.059	1.256

Las medias de validación no son cero por la misma razón que en #54: 2025 termina el 30 de junio, así que le sobra primavera y le falta otoño. El desplazamiento es el calendario, no un error de ajuste.

```

PUNT.to_parquet(PROC / "af_puntajes_diarios.parquet", index=False)
cargas.to_csv(PROC / "af_cargas.csv")
pd.DataFrame(W, index=NOMBRES, columns=PRED).to_csv(PROC / "af_pesos.csv")
print(f"guardados {len(PUNT):,} puntajes, cargas {cargas.shape} y pesos
      ↪ {W.shape}")

```

guardados 17,399 puntajes, cargas (13, 5) y pesos (3, 13)

7. Cierre

Tres factores, elegidos por MAP de Velicer tras descartar el análisis paralelo por inflación con n , explican el 40.4 % de la varianza de trece predictores y admiten lectura física completa: combustión primaria, ventilación y forzamiento fotoquímico. Los tres ejes que #54 encontró aparecen otra vez por un camino distinto, con estructura simple más limpia — cada variable carga en un factor y casi en ninguno más.

Dos resultados que el PCA no podía entregar:

1. **El eje de estancamiento era un eje de viento.** La presión barométrica tiene una comunalidad de 0.134, así que casi toda su varianza es específica. El nombre *estancamiento atmosférico* de #54 atribuía a estructura común algo que el modelo identifica como varianza única.
2. **La redundancia dejó de ser invisible.** El PCA absorbió NOX y RH_min_diurna sin protestar; el modelo factorial devolvió comunalidades imposibles hasta que se podaron. Las trece variables que quedan tienen un VIF máximo de 2.54.

Los puntajes de Bartlett tienen VIF de 1.01, así que **el análisis factorial puede sustituir al PCA también en su papel instrumental** y no solo en el interpretativo. Lo que #55 consumiría es `af_puntajes_diarios.parquet` en lugar de `pca_puntuaciones_diarias.parquet`.

Queda declarado como limitación lo que no mejoró: el KMO de 0.679 sigue siendo mediocre, el RMSR de 0.055 pasa la convención sin holgura y 24 de 78 residuos exceden 0.05. Y persiste la fuga

no supervisada de #52 —el de Box-Cox se estimó sobre el periodo completo— idéntica a la que #54 declaró y por la misma razón: revertirla exigiría rehacer el dataset transformado.

Sin mediciones de compuestos orgánicos volátiles, los tres factores se describen como **consistentes con** combustión, ventilación y fotoquímica; no hay base para afirmar un mecanismo ni para atribuir la formación de ozono a un régimen NO_x o COV.